

International Institute of Information Technology, Bangalore
Software Production Engineering (SPE)

Final Project Report

Doctor-Patient Appointment Management System

A Cloud-Native Microservices Application with Full DevOps CI/CD Pipeline

Student Name	Roll Number
Parag Piprewar	MT2025083
Raghav Goyal	MT2025098

May 2026

Github repo link : <https://github.com/ParagRider1/Healthcare>

DockerHub (All Services)

<https://hub.docker.com/u/paragrider1>

1. Introduction

This project implements a Doctor-Patient Appointment Management System using a cloud-native microservices architecture. The system enables patients to register, doctors to manage their availability, and appointments to be booked and confirmed via automated email notifications. The entire system is designed to replicate real-world healthcare service workflows in a modular, fault-tolerant, and scalable manner.

The primary objective of this project is to demonstrate end-to-end DevOps practices — from source code management and containerization to continuous integration, automated deployment, monitoring, and secrets management — using industry-standard tools.

1.1 Objectives

- Design and implement a healthcare system using Spring Boot microservices.
- Containerize all services using Docker and Docker Compose for local testing.
- Set up a CI/CD pipeline using Jenkins triggered by GitHub webhooks.
- Deploy the system to a Kubernetes cluster (Minikube) using Ansible playbooks.
- Monitor system health using Prometheus and Grafana.
- Centralize logs using the ELK Stack (Elasticsearch, Logstash, Kibana) with Filebeat.
- Manage secrets securely using HashiCorp Vault.
- Demonstrate horizontal auto-scaling using Kubernetes HPA.

1.2 Technology Stack

Layer	Technology
Language & Framework	Java 17, Spring Boot 3.x, Spring Cloud
Service Registry	Netflix Eureka (Spring Cloud Netflix)
API Gateway	Spring Cloud Gateway
Containerization	Docker, Docker Compose
Orchestration	Kubernetes (Minikube), Kubectl
CI/CD	Jenkins (Pipeline as Code via Jenkinsfile)
Automation	Ansible (Playbooks + Roles)
Database	MySQL 5.7 (per-service database)
Message Broker	RabbitMQ
Monitoring	Prometheus, Grafana
Logging	Elasticsearch, Logstash, Kibana (ELK), Filebeat
Secrets Management	HashiCorp Vault
Container Registry	Docker Hub (paraglider1)

Microservices

- **Patient Service** – Handles CRUD operations for patient profiles.
- **Doctor Service** – Manages doctor data and availability.
- **Appointment Service** – Coordinates appointments between doctors and patients.
- **Notification Service** – Sends out confirmation emails using Gmail and OAuth2.

Tech Stack

- **Backend:** Java 17, Spring Boot, Spring Cloud
- **Containerization:** Docker, Docker Compose
- **Orchestration:** Kubernetes with Deployments, Services, Ingress, ConfigMaps
- **Monitoring:** Prometheus for scraping, Grafana for visualization
- **CI/CD:** Jenkins + GitHub Webhook + Ansible for K8s deployment
- **Database:** MySQL (each service has its own DB)
- **Broker:** RabbitMQ for asynchronous messaging

CI/CD Workflow

1. **Code Commit** – Developer pushes code to GitHub.
2. **Jenkins Trigger** – Webhook initiates Jenkins build.
3. **Docker Build** – Jenkins builds Docker images and pushes to Docker Hub.
4. **Ansible Deploy** – Ansible playbook deploys YAML files to Kubernetes.
5. **Monitoring Active** – Metrics collected by Prometheus, visualized on Grafana.

2. System Architecture

The system is built on a microservices architecture where each service owns its domain and data. Services communicate with each other via REST APIs (synchronously) and via RabbitMQ (asynchronously). All services register themselves with the Eureka Service Registry, and all external traffic is routed through the Spring Cloud API Gateway.

The high-level architecture follows a three-tier pattern: the CI/CD layer (GitHub + Jenkins + Ansible), the application layer (Kubernetes-hosted microservices), and the observability layer (Prometheus, Grafana, ELK Stack, HashiCorp Vault).

2.1 Architecture Layers

Layer	Components	Role
-------	------------	------

CI/CD	GitHub, Jenkins, Ansible	Source control, build automation, deployment
API Layer	Spring Cloud Gateway	Single entry point, routing, load balancing
Application	4 Spring Boot Microservices	Business logic for each domain
Messaging	RabbitMQ	Asynchronous appointment notifications
Data	MySQL (per-service)	Isolated data stores for each service
Monitoring	Prometheus + Grafana	Metrics collection and visualization
Logging	ELK Stack + Filebeat	Centralized log aggregation
Secrets	HashiCorp Vault	Secure secrets management

3. Microservices Design

The system consists of four domain microservices, plus two infrastructure services (Service Registry and API Gateway). Each service is independently deployable, has its own database, and communicates via well-defined APIs.

3.1 Service Registry (Eureka Server) — Port 8761

The Eureka Server acts as the service discovery hub. All microservices register their IP addresses and ports with Eureka on startup. When one service needs to call another, it queries Eureka for the target service's location instead of using hardcoded IP addresses. This enables dynamic scaling and resilience.

3.2 API Gateway — Port 8765

The Spring Cloud API Gateway is the single entry point for all client requests. It routes requests to the appropriate downstream microservice based on the URL path. It integrates with Eureka for dynamic route resolution and supports load balancing across multiple replicas.

3.3 Patient Service — Port 8081

Manages all patient-related CRUD operations. Each patient is stored in the dedicated patientdb MySQL database.

Endpoint	Description
POST /patient/api/patients	Register a new patient
GET /patient/api/patients	List all patients
GET /patient/api/patients/{id}	Get patient by ID
PUT /patient/api/patients/{id}	Update patient
DELETE /patient/api/patients/{id}	Delete patient

3.4 Doctor Service — Port 8082

Handles doctor profile management and availability. Connected to doctordb MySQL database.

Endpoint	Description
POST /doctor/api/doctors	Register a doctor
GET /doctor/api/doctors	List all doctors
GET /doctor/api/doctors/{id}	Get doctor by ID
PUT /doctor/api/doctors/{id}	Update doctor profile

3.5 Appointment Service — Port 8083

Coordinates the booking of appointments between patients and doctors. Uses OpenFeign to call Patient and Doctor services for validation. Upon successful booking, it publishes a message to RabbitMQ.

Endpoint	Description
POST /appointment/api/appointments	Book an appointment
GET /appointment/api/appointments	List all appointments
GET /appointment/api/appointments/{id}	Get appointment by ID

Async configuration: Publishes to appointment.exchange with routing key appointment.email. Connected to appointmentdb MySQL database.

3.6 Notification Service — Port 8085

Listens to RabbitMQ queue appointment.notification.queue and sends confirmation emails using JavaMailSender configured with Gmail OAuth2. It is a purely event-driven service with no direct REST endpoints for booking.

Configuration	Value
Queue	appointment.notification.queue
Exchange	appointment.exchange (Direct)
Routing Key	appointment.email
Email Sender	JavaMailSender via Gmail OAuth2

4. Containerization with Docker

4.1 Docker Images

Each microservice has its own Dockerfile to build a self-contained, runnable image. All images are published to Docker Hub under the paragrider1 namespace.

Service	Docker Hub Image
Service Registry	paragrider1/service-registry
API Gateway	paragrider1/api-gateway
Patient Service	paragrider1/patient-service
Doctor Service	paragrider1/doctor-service
Appointment Service	paragrider1/appointment-service
Notification Service	paragrider1/notification-service

4.2 Docker Hub — All Services Published

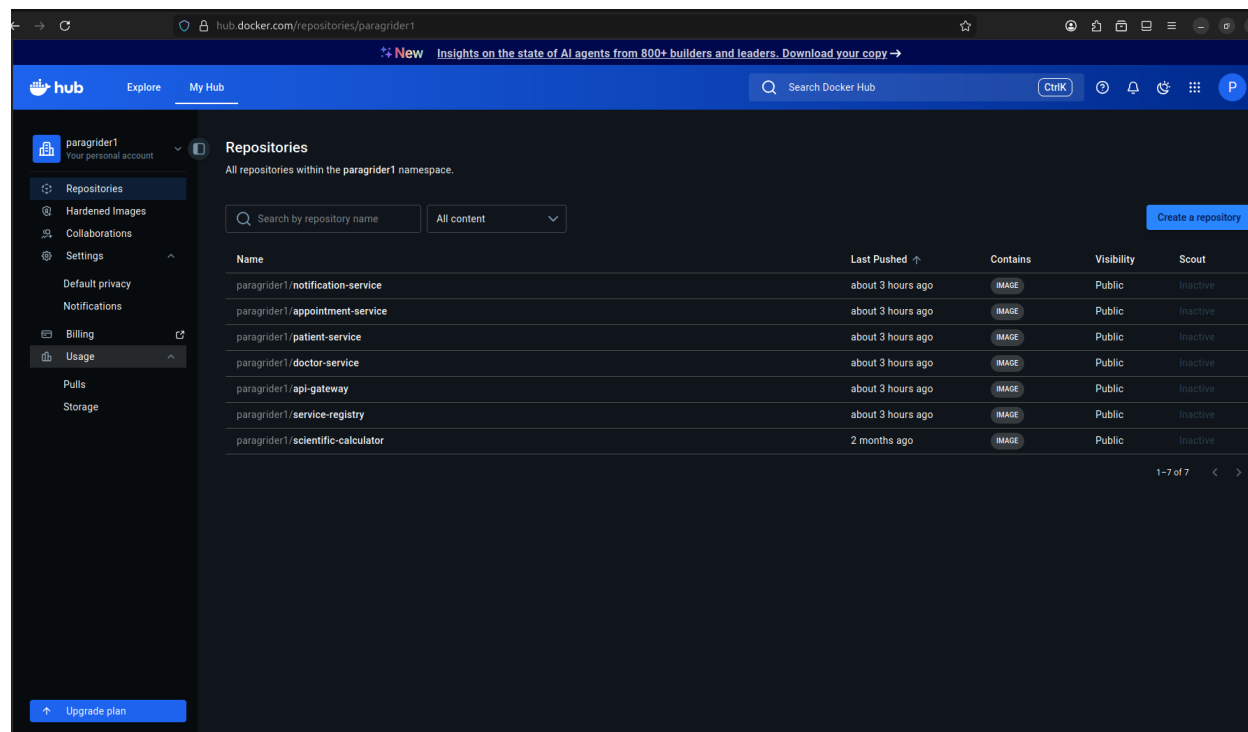


Figure 1: Docker Hub — All 6 microservice images published under paragrider1 namespace (pushed ~3 hours ago)

4.3 Docker Compose (Local Testing)

Two Docker Compose files are provided for local testing:

- `docker-compose.yml` — Runs core microservices (Eureka, Gateway, MySQL, RabbitMQ, Patient, Doctor, Appointment, Notification services).
- `docker-compose-elk.yml` — Extends the main compose file to add Elasticsearch, Logstash, Kibana, Filebeat, and HashiCorp Vault.

To start locally:

```
# Start core services
docker compose up -d

# Start ELK + Vault on top
docker compose -f docker-compose.yml -f docker-compose-elk.yml up -d \
  elasticsearch logstash kibana filebeat vault

# Verify all containers
docker compose ps
```


5. CI/CD Pipeline with Jenkins

5.1 Pipeline Overview

Jenkins is installed locally and configured to trigger automatically via a GitHub Webhook on every git push to the main branch. The pipeline is defined as code in a Jenkinsfile at the root of the repository.

Stage	Description	Duration
Declarative: Checkout SCM	Jenkins clones the latest code from GitHub	~2s
Checkout	Verifies Git branch and commit	~1s
Build Maven JARs	Compiles Java source for all 6 services with mvn clean package -DskipTests	~1m 9s
Run Tests	Executes unit tests for Patient and Doctor services	~4s
Build Docker Images	Builds Docker images for all 6 services	~20s
Push to DockerHub	Pushes all images to paraglider1/* on Docker Hub	~6m 38s
Apply K8s Manifests with Ansible	Runs the Ansible playbook to deploy to Kubernetes	~1m 36s
Health Check	Runs kubectl get pods, services, hpa	~33s

5.2 Jenkins Pipeline — Stage View (Build #10)

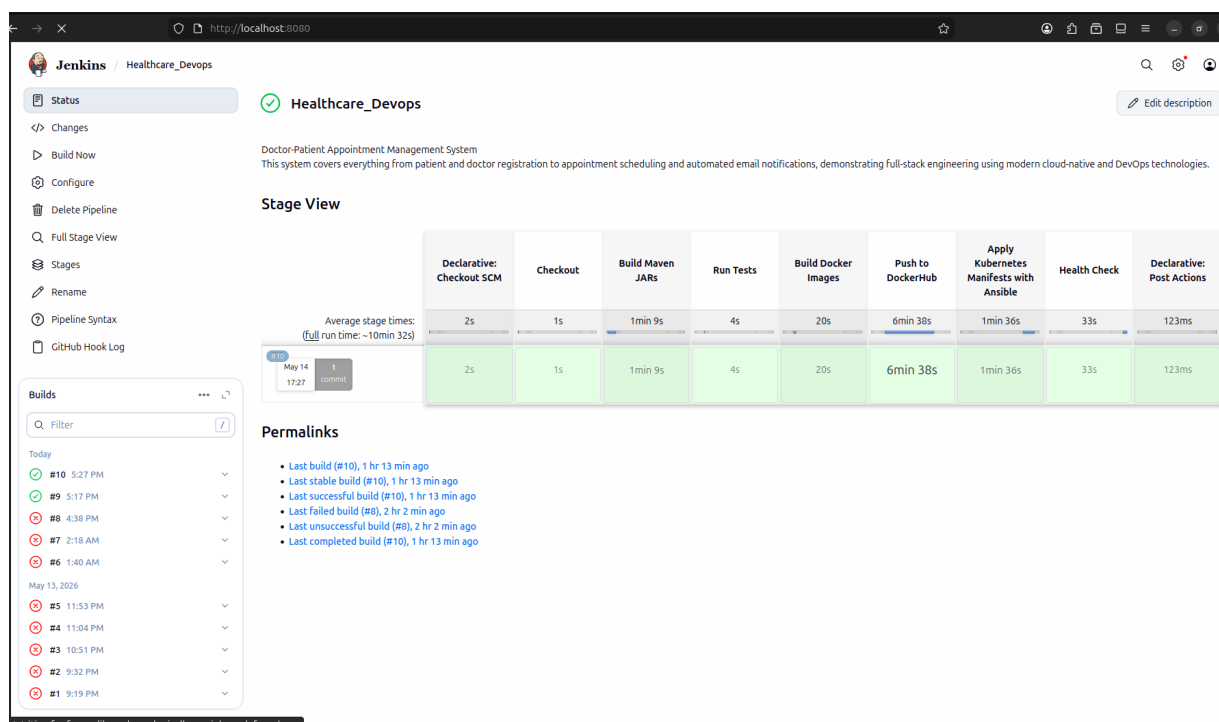


Figure 2: Jenkins Healthcare_Devops pipeline — Build #10 successful with all 9 stages completed. Total run time: ~10 minutes 32 seconds.

5.3 Jenkinsfile (Pipeline as Code)

```
pipeline {
    agent any
    triggers {
        githubPush()
    }

    environment {
        GITHUB_REPO_URL = 'https://github.com/ParagRider1/Healthcare.git'
        EUREKA_DOCKER_IMAGE = 'paragrider1/service-registry'
        GATEWAY_DOCKER_IMAGE = 'paragrider1/api-gateway'
        DOCTOR_DOCKER_IMAGE = 'paragrider1/doctor-service'
        PATIENT_DOCKER_IMAGE = 'paragrider1/patient-service'
        APPOINTMENT_DOCKER_IMAGE = 'paragrider1/appointment-service'
        NOTIFICATION_DOCKER_IMAGE = 'paragrider1/notification-service'
        KUBECONFIG = '/var/lib/jenkins/.kube/config'
    }

    stages {
        stage('Checkout') {
            steps {
                script {
                    git branch: 'main', url: "${GITHUB_REPO_URL}"
                }
            }
        }
        stage('Build Maven JARs') {
            steps {
                script {
                    dir('ServiceRegistry') {
                        sh 'mvn clean package -DskipTests'
                    }
                    dir('ApiGateway') {
                        sh 'mvn clean package -DskipTests'
                    }
                    dir('DoctorService') {
                        sh 'mvn clean package -DskipTests'
                    }
                    dir('PatientService') {
                        sh 'mvn clean package -DskipTests'
                    }
                }
            }
        }
    }
}
```

```
    }
    dir('AppointmentService') {
        sh 'mvn clean package -DskipTests'
    }
    dir('NotificationService') {
        sh 'mvn clean package -DskipTests'
    }
}

stage('Run Tests') {
    steps {
        script {
            dir('PatientService') {
                sh 'mvn test -pl PatientService || true'
            }
            dir('DoctorService') {
                sh 'mvn test -pl DoctorService || true'
            }
        }
    }
}

stage('Build Docker Images') {
    steps {
        script {
            def eurekaImage = docker.build("${EUREKA_DOCKER_IMAGE}",
            './ServiceRegistry/')
            def gatewayImage = docker.build("${GATEWAY_DOCKER_IMAGE}",
            './ApiGateway/')
            def doctorImage = docker.build("${DOCTOR_DOCKER_IMAGE}",
            './DoctorService/')
            def patientImage = docker.build("${PATIENT_DOCKER_IMAGE}",
            './PatientService/')
            def appointmentImage = docker.build("${
            APPOINTMENT_DOCKER_IMAGE}", './AppointmentService/')
            def notificationImage = docker.build("${
            NOTIFICATION_DOCKER_IMAGE}", './NotificationService/')
        }
    }
}

stage('Push to DockerHub') {
```

```
    steps {
      script {
        docker.withRegistry('', 'dockerhub-credentials') {
          docker.image("${EUREKA_DOCKER_IMAGE}").push()
          docker.image("${GATEWAY_DOCKER_IMAGE}").push()
          docker.image("${DOCTOR_DOCKER_IMAGE}").push()
          docker.image("${PATIENT_DOCKER_IMAGE}").push()
          docker.image("${APPOINTMENT_DOCKER_IMAGE}").push()
          docker.image("${NOTIFICATION_DOCKER_IMAGE}").push()
        }
      }
    }
  }
}

stage('Apply Kubernetes Manifests with Ansible') {
  steps {
    script {
      withEnv(["ANSIBLE_HOST_KEY_CHECKING=False"]) {
        ansiblePlaybook(
          playbook: 'ansible-playbook.yaml',
          inventory: 'inventory.ini',
          vaultCredentialsId: 'ansible_vault_pass'
        )
      }
    }
  }
}

stage('Health Check') {
  steps {
    script {
      sh 'sleep 30'
      sh 'kubectl get pods -o wide || true'
      sh 'kubectl get services || true'
      sh 'kubectl get hpa || true'
    }
  }
}
}
```

6. Kubernetes Orchestration

6.1 Kubernetes Manifests

The Kubernetes/ directory contains 17 manifest files that define the complete production deployment including all microservices, infrastructure, monitoring, logging, and secrets management components.

Manifest File	Resources Defined
pvc.yml	PersistentVolumeClaim for MySQL data
mysql.yml	MySQL Deployment, Service, PersistentVolume
rabbitmq.yml	RabbitMQ Deployment + Service (NodePort)
eureka.yml	Eureka Deployment + Service (NodePort :30061)
gateway.yml	API Gateway Deployment + Service (NodePort :30065)
patient.yml	PatientService Deployment + Service + HPA
doctor.yml	DoctorService Deployment + Service + HPA
appointment.yml	AppointmentService Deployment + Service + HPA
notification.yml	NotificationService Deployment + Service + HPA
prometheus.yml	Prometheus Deployment + ConfigMap + Service
grafana.yml	Grafana Deployment + Service (NodePort :30030)
elk.yml	Elasticsearch + Logstash + Kibana Deployments & Services
filebeat.yml	Filebeat DaemonSet + RBAC roles
vault.yml	HashiCorp Vault Deployment + Service (NodePort :30820)
healthcare-ingress.yml	NGINX Ingress routing all services via healthcare.local

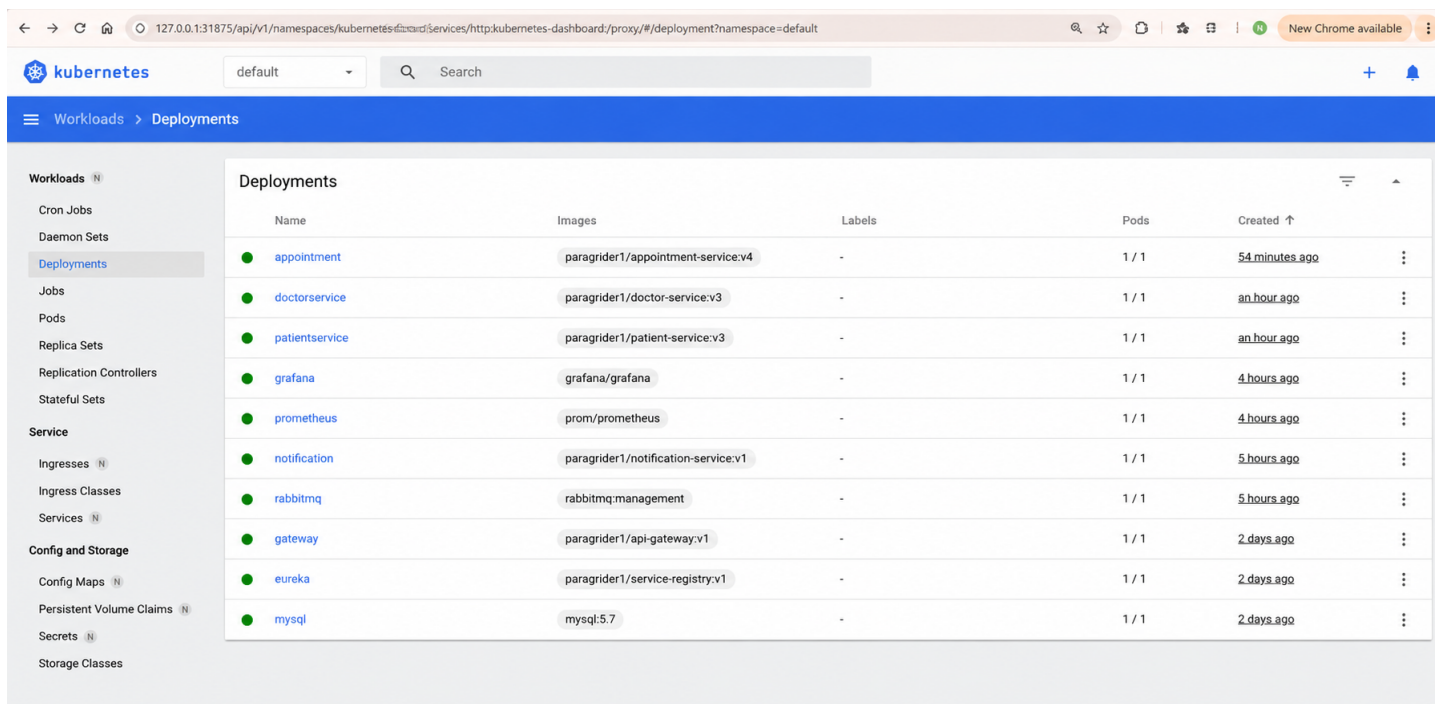
6.2 Kubernetes Services

```

parag-piprewar@parag-piprewar-HP-Pavilion-x360-Convertible-14-dw1xxx: ~/Downloads/Healthcare-main
get hpa
NAME                REFERENCE                TARGETS          MINPODS   MAXPODS   REPLICAS   AGE
appointment-hpa     Deployment/appointment    cpu: 2%/50%     1         2         1          6h48m
doctorservice-hpa   Deployment/doctorservice  cpu: 2%/50%     1         2         1          6h48m
notification-hpa    Deployment/notification   cpu: 2%/50%     1         2         1          6h48m
patientservice-hpa  Deployment/patientservice  cpu: 2%/50%     1         2         1          6h48m
parag-piprewar@parag-piprewar-HP-Pavilion-x360-Convertible-14-dw1xxx:~/Downloads/Healthcare-main$ minikube service kibana --url
http://192.168.49.2:30561
parag-piprewar@parag-piprewar-HP-Pavilion-x360-Convertible-14-dw1xxx:~/Downloads/Healthcare-main$ kubectl get svc
NAME                TYPE                CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
appointment         ClusterIP            10.96.77.101    <none>           8083/TCP         7h6m
doctorservice        ClusterIP            10.99.9.245     <none>           8082/TCP         7h6m
elasticsearch        ClusterIP            10.108.106.218  <none>           9200/TCP,9300/TCP 7h6m
eureka              NodePort             10.102.144.63   <none>           8761:30061/TCP   7h21m
gateway             NodePort             10.104.233.191  <none>           8765:30065/TCP   7h21m
grafana             NodePort             10.106.168.49   <none>           3000:30030/TCP   7h6m
kibana              NodePort             10.108.6.114    <none>           5601:30561/TCP   7h6m
kubernetes           ClusterIP            10.96.0.1       <none>           443/TCP          7h23m
logstash            ClusterIP            10.109.202.223  <none>           5044/TCP,9600/TCP 7h6m
mysql               ClusterIP            10.111.17.205   <none>           3306/TCP         7h21m
notification         ClusterIP            10.106.96.178   <none>           8085/TCP         7h6m
patientservice       ClusterIP            10.98.110.191   <none>           8081/TCP         7h6m
prometheus          NodePort             10.109.65.34    <none>           9090:30090/TCP   7h6m
rabbitmq            NodePort             10.99.60.107    <none>           5672:30072/TCP,15672:31672/TCP 7h6m
parag-piprewar@parag-piprewar-HP-Pavilion-x360-Convertible-14-dw1xxx:~/Downloads/Healthcare-main$

```

Figure 3: `kubectl get svc` — all microservices and infrastructure services running in Kubernetes with assigned ClusterIPs and NodePorts



6.3 Horizontal Pod Autoscaler (HPA)

All four microservices are configured with Kubernetes HPA using CPU utilization as the scaling metric. Each service scales between 1 and 2 replicas at 50% CPU utilization.

```

parag-piprewar@parag-piprewar-HP-Pavilion-x360-Convertible-14-dw1xxx:~/Downloads/Healthcare-main$ kubectl get hpa
NAME                REFERENCE                TARGETS          MINPODS   MAXPODS
appointment-hpa     Deployment/appointment    cpu: 2%/50%     1         2
doctorservice-hpa   Deployment/doctorservice  cpu: 2%/50%     1         2
notification-hpa    Deployment/notification   cpu: 2%/50%     1         2
patientservice-hpa  Deployment/patientservice  cpu: 2%/50%     1         2

```

Figure 4: `kubectl get hpa` — all 4 HPAs active, tracking CPU at 2%/50% with min=1 max=2 replicas

HPA Name	Min Replicas	Max Replicas	CPU Threshold
patientservice-hpa	1	2	50%
doctorservice-hpa	1	2	50%
appointment-hpa	1	2	50%
notification-hpa	1	2	50%

7. Ansible Automation

7.1 Ansible Playbook Structure

The deployment is fully automated using Ansible. A single playbook orchestrates 8 roles, each responsible for deploying a specific layer of the system. Sensitive values are encrypted using Ansible Vault and stored in secrets.yml.

```
---
- name: Apply Kubernetes manifests using Ansible roles
  hosts: localhost
  remote_user: parag-piprewar
  vars:
    kubeconfig_path: /home/parag-piprewar/.kube/config
  roles:
    - role: envSetup_role      # MySQL, Eureka, Gateway, Ingress
    - role: patient_role      # Patient microservice
    - role: doctor_role       # Doctor microservice
    - role: appointment_role  # Appointment microservice
    - role: notification_role  # RabbitMQ + Notification service
    - role: monitoring_role   # Prometheus + Grafana
    - role: elk_role          # ELK Stack + Filebeat
    - role: vault_role        # HashiCorp Vault
```

7.2 Ansible Roles

Role	Responsibilities
envSetup_role	Applies PVC, MySQL ConfigMap, MySQL, Eureka, Gateway, Ingress
patient_role	Deploys Patient Service
doctor_role	Deploys Doctor Service
appointment_role	Deploys Appointment Service
notification_role	Deploys RabbitMQ + Notification Service
monitoring_role	Deploys Prometheus + Grafana
elk_role	Deploys ELK Stack + Filebeat, waits for readiness (300s timeout)
vault_role	Deploys HashiCorp Vault

7.3 Ansible Vault Encryption

```
# Encrypt secrets file
ansible-vault encrypt secrets.yml

# Run playbook with vault password
ansible-playbook -i inventory.ini ansible-playbook.yaml --ask-vault-pass
```

In Jenkins, the vault password is stored as credential `ansible_vault_pass` and passed to the Ansible plugin automatically, enabling fully automated encrypted deployments.

8. Monitoring with Prometheus & Grafana

Monitoring & Logs

Prometheus + Grafana

- Scrapes actuator metrics from each service
- **Tracks heap memory, thread count, HTTP response times, et**

8.1 Prometheus Scrape Configuration

Each microservice exposes metrics via the Spring Boot Actuator endpoint at `/actuator/prometheus`. Prometheus scrapes these endpoints at regular intervals to collect JVM and application-level metrics.

```
scrape_configs:
  - job_name: 'backend monitoring'
    metrics_path: '/actuator/prometheus'
    static_configs:
      - targets:
        - 'patientservice:8081'
        - 'doctorservice:8082'
        - 'appointment:8083'
        - 'notification:8085'
```

8.2 Prometheus Targets — All Services UP

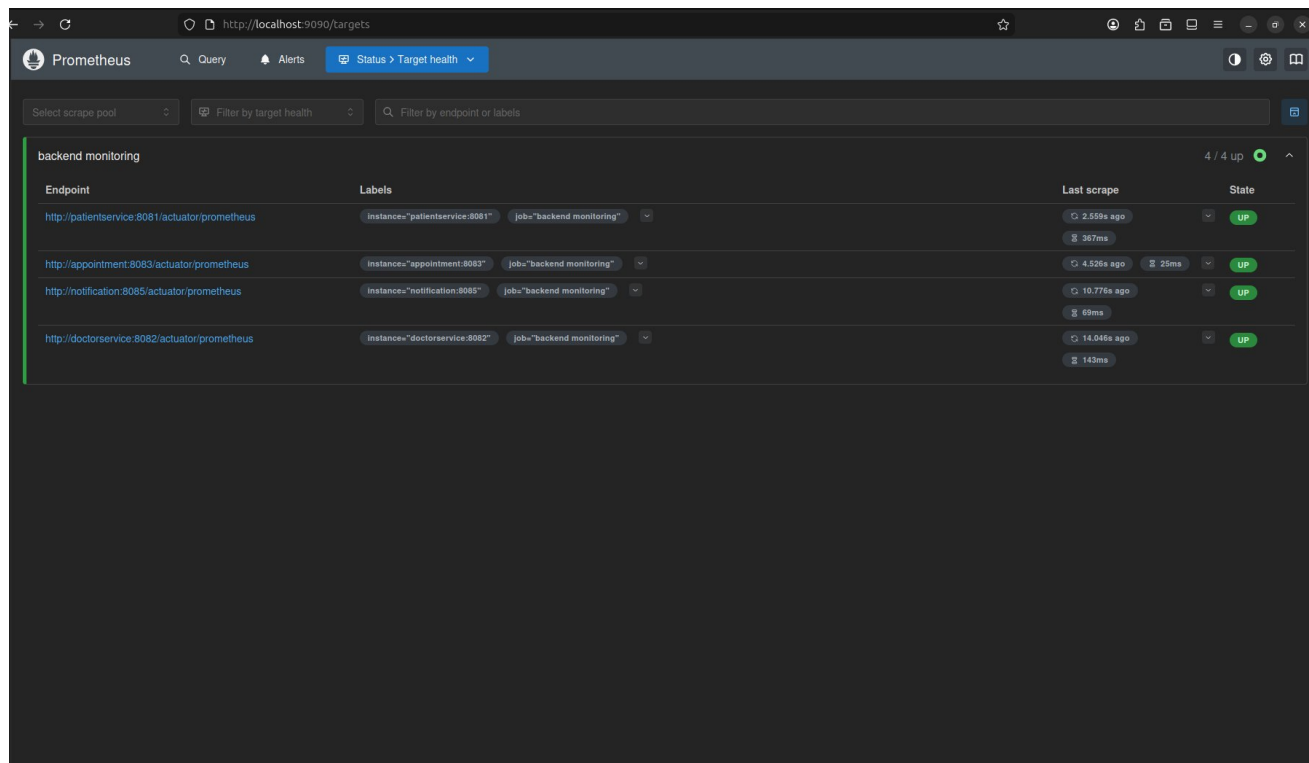


Figure 5: Prometheus Target Health — all 4 microservices (patient :8081, appointment :8083, notification :8085, doctor :8082) are UP with recent scrape timestamps

8.3 Grafana Dashboard

Grafana is accessible at `http://localhost:3000`. Dashboards track JVM heap memory, HTTP request latency, thread counts, HikariCP connection pool usage, and RabbitMQ message rates.

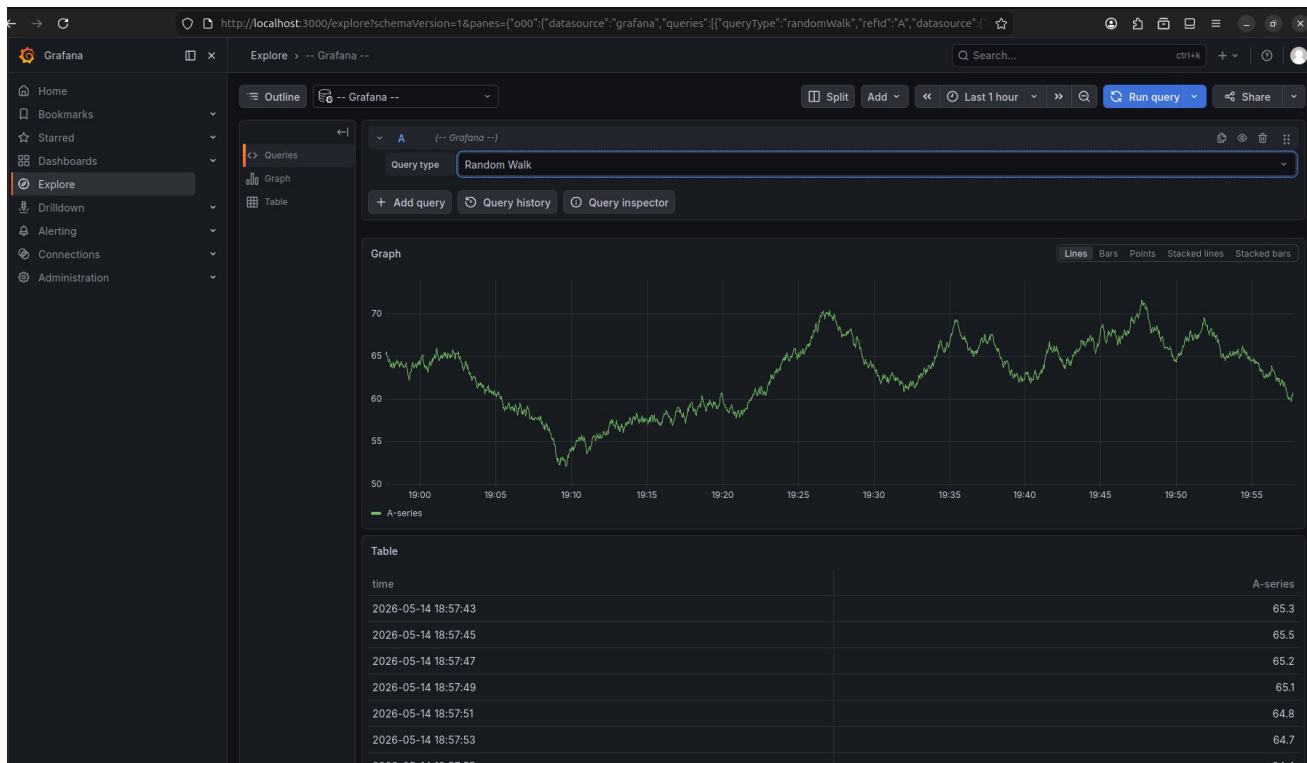
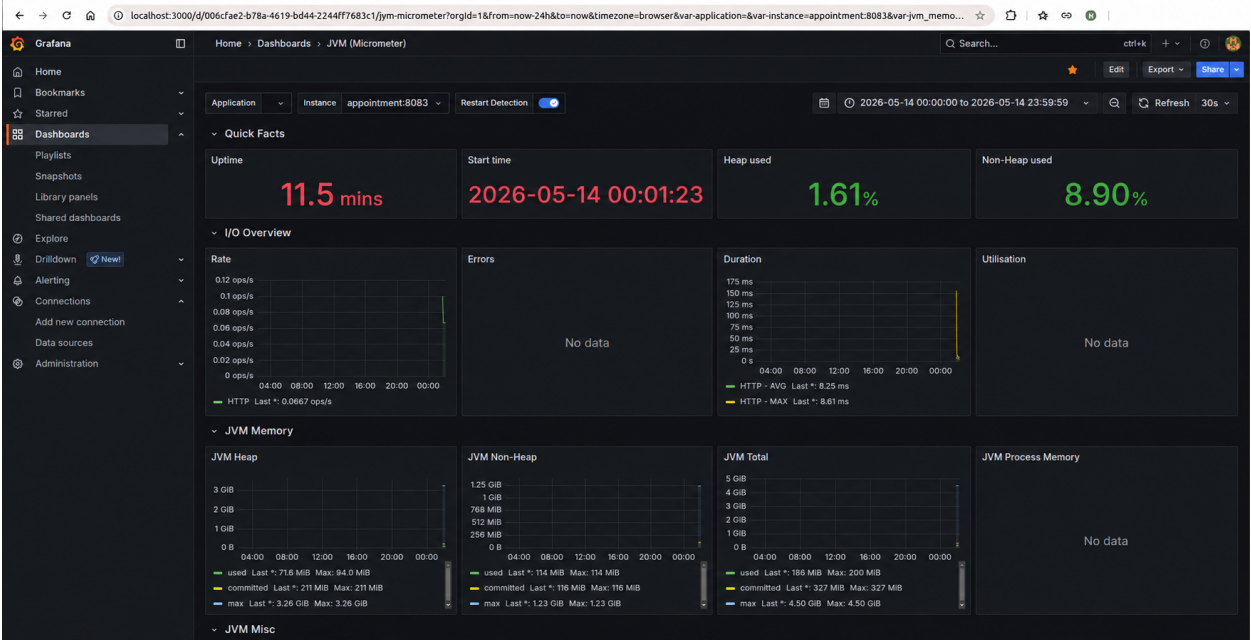


Figure 6: Grafana Explore — real-time time-series metrics from healthcare services showing A-series data over the last hour





9. Centralized Logging with ELK Stack

9.1 Architecture

Logs from all microservice pods are collected and forwarded through the following pipeline:

Source	Collector	Processor	Destination
Microservice Pods	Filebeat DaemonSet	Logstash :5044	Elasticsearch :9200 → Kibana :5601

9.2 Logstash Pipeline

```
input { beats { port => 5044 } }

filter {
  grok {
    match => {
      "message" => "%{TIMESTAMP_ISO8601:timestamp} %{LOGLEVEL:log_level}
                  %{DATA:logger} - %{GREEDYDATA:log_message}"
    }
  }
}

output {
  elasticsearch {
    hosts => ["http://elasticsearch:9200"]
    index => "healthcare-logs-%{+YYYY.MM.dd}"
  }
}
```

9.3 Elasticsearch Init Container

To prevent Elasticsearch from crashing due to the `vm.max_map_count` kernel limit, an init container is used:

```
initContainers:
- name: increase-vm-max-map
  image: busybox
  command: ["sysctl", "-w", "vm.max_map_count=262144"]
  securityContext:
    privileged: true
```

10. Secrets Management with HashiCorp Vault

10.1 Vault in Kubernetes

HashiCorp Vault is deployed in development mode inside the Kubernetes cluster. In production, this would be replaced with a HA Vault cluster with proper unseal mechanisms.

```
containers:
  - name: vault
    image: hashicorp/vault:1.15
    command:
      - vault
      - server
      - -dev
      - -dev-root-token-id=root
      - -dev-listen-address=0.0.0.0:8200
```

10.2 Seeding Secrets

```
kubect1 port-forward svc/vault 8200:8200 &
export VAULT_ADDR=http://localhost:8200
export VAULT_TOKEN=root

vault kv put secret/healthcare/mysql username=root password=admin
vault kv put secret/healthcare/rabbitmq username=guest password=guest
vault kv get secret/healthcare/mysql
```

11. Asynchronous Communication with RabbitMQ

The Appointment Service publishes events to RabbitMQ when a new appointment is booked. The Notification Service consumes these events and sends confirmation emails via Gmail OAuth2 — fully decoupled and asynchronous.

11.1 Message Flow

- AppointmentService books an appointment and saves it to appointmentdb.
- AppointmentService publishes a message to appointment.exchange with routing key appointment.email.
- RabbitMQ routes the message to appointment.notification.queue.
- NotificationService receives the message and sends a Gmail confirmation email using JavaMailSender with OAuth2.

11.2 RabbitMQ Configuration

Parameter	Value
Exchange	appointment.exchange (Direct)
Queue	appointment.notification.queue
Routing Key	appointment.email
Management UI	http://localhost:15672 (guest/guest)

11.3 RabbitMQ Management UI

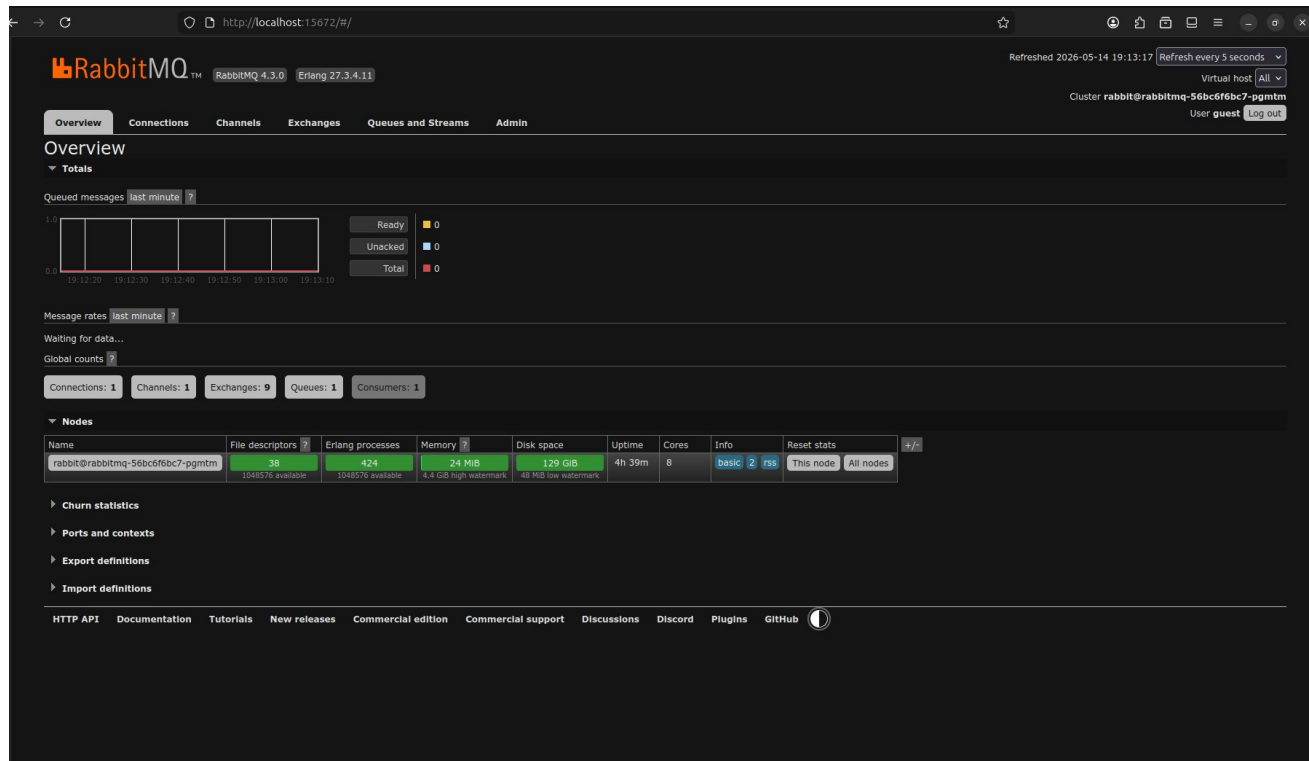


Figure 7: RabbitMQ 4.3.0 Management UI — healthy cluster with 1 connection, 9 exchanges, 1 queue, 1 consumer, running on Erlang 27.3.4.11

12. Service Discovery with Eureka

All microservices register with the Spring Eureka server at startup, enabling dynamic service discovery and load-balanced inter-service communication without hardcoded hostnames.

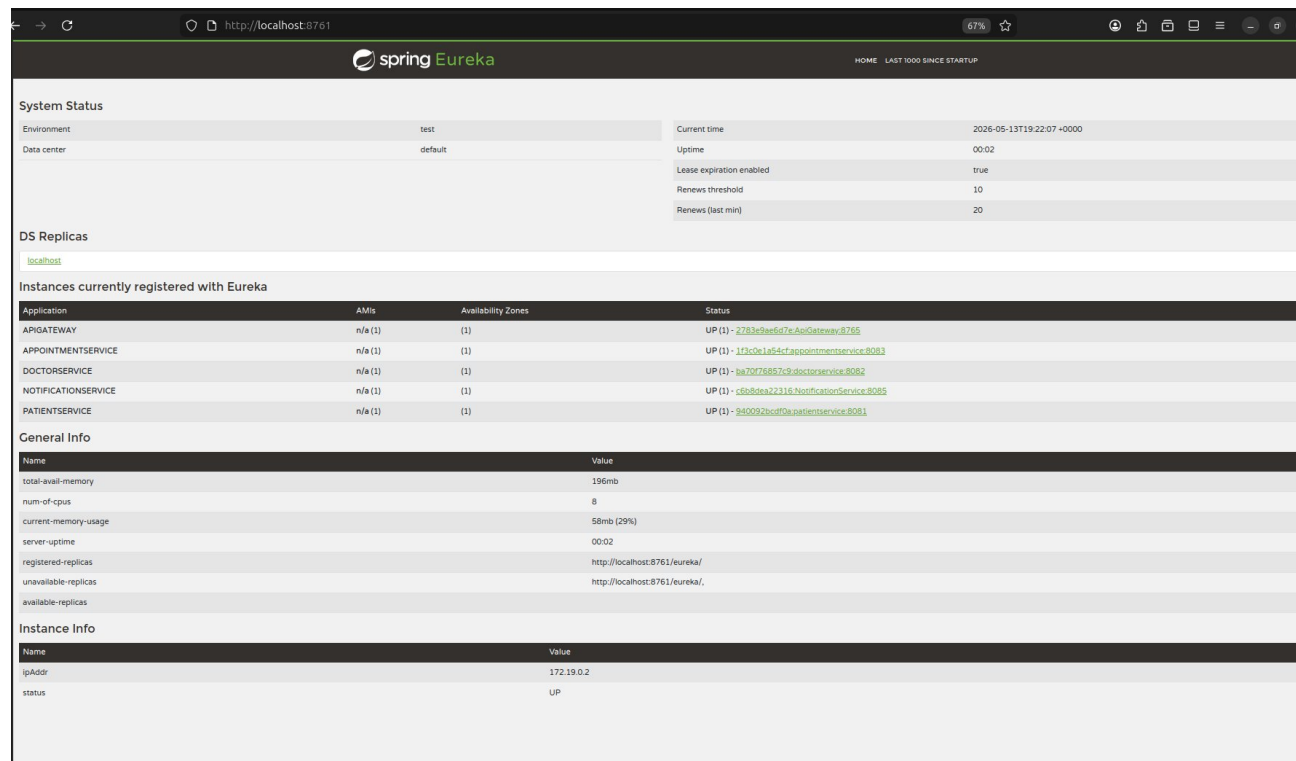


Figure 8: Spring Eureka Dashboard — APIGATEWAY, APPOINTMENTSERVICE, DOCTORSERVICE, NOTIFICATIONSERVICE, and PATIENTSERVICE all registered and UP (status: UP 1/1)

13. Conclusion

This project successfully demonstrates a complete end-to-end DevOps pipeline for a production-grade healthcare microservices application. All objectives were met, including microservice design, containerization, CI/CD, Kubernetes orchestration, auto-scaling, monitoring, centralized logging, secrets management, and asynchronous messaging.

Criteria	Status	Details
Git + GitHub	DONE	ParagRider1/Healthcare, incremental commits
Jenkins CI/CD + GitHub Webhook	DONE	9-stage pipeline, Build #10 success (~10m 32s)
Docker + Docker Compose	DONE	6 services containerized, published to paragrider1
Ansible Playbooks + 8 Roles	DONE	Modular roles, ansible-vault encryption
Kubernetes + Ingress	DONE	17 manifests, NGINX ingress, NodePort services
HPA Auto-scaling	DONE	CPU-based scaling on all 4 microservices
Prometheus + Grafana	DONE	All 4 targets UP, real-time dashboards
ELK Centralized Logging	DONE	Filebeat → Logstash → Elasticsearch → Kibana
HashiCorp Vault	DONE	Deployed in Kubernetes, secrets seeded
RabbitMQ Async Messaging	DONE	Appointment → Notification email flow

13.1 Key Learnings

1. Resource Planning: A full DevOps stack (ELK + 6 Spring Boot apps + databases) in Minikube requires careful CPU and memory limit tuning per container.
2. Ansible Roles: Modular role structure makes automation easily maintainable and independently testable.
3. GitOps Principle: Storing infrastructure-as-code alongside application code enables reproducible, auditable deployments.
4. Observability: The combination of Prometheus + Grafana + ELK Stack provides complete 360-degree visibility — metrics, logs, and events.

GitHub Repository: <https://github.com/ParagRider1/Healthcare>

Docker Hub: <https://hub.docker.com/u/paragrider1>

Report by: MT2025083 Parag Piprewar | MT2025098 Raghav Goyal

International Institute of Information Technology, Bangalore — May 2026